年画智能系统 - Strands Agent 本地部署方案

📋 项目概述

基于 Strands Agents 框架,串接四个AI模块构建年画智能系统。

四个核心模块

1. **年画QA模块** - Qwen2.5-VL-3B (多模态问答)
2. **用户标签分析** - Qwen3-4B (用户行为分析)
3. **智能推荐** - Qwen3-4B (个性化推荐)
4. **图像生成** - Stable Diffusion 1.5 LoRA (年画生成)

🏗️ 系统架构设计

架构模式

采用 **Hierarchical (层级式)** 多Agent模式:

```
用户输入
    ↓
主协调器 (Orchestrator Agent)
    ├── 理解用户意图
    ├── 编排任务流程
    ├── 调用子模块
    └── 整合返回结果

四个子Agent作为工具:
├─ QA Agent (回答年画知识)
├─ 标签Agent (分析用户偏好)
├─ 推荐Agent (推荐年画作品)
└─ 生成Agent (生成年画图像)
```

工作流程

单一任务流程: 用户提问 → 主协调器识别意图 → 调用对应Agent → 返回结果

复杂任务流程 (例如: "分析我的偏好并推荐作品"):

1. 主协调器识别这是多步骤任务
2. 先调用标签Agent分析用户偏好
3. 将分析结果传给推荐Agent
4. 整合两个Agent的输出返回用户

关键特点: 主协调器由LLM驱动,自动判断需要调用哪些Agent,以及调用顺序,无需手动编写复杂的工作流逻辑。

⚙️ 环境准备

1. 硬件要求

- **GPU**: NVIDIA显卡,至少48GB显存
- **内存**: 32GB+
- **硬盘**: 50GB+ 可用空间
- **操作系统**: Linux/macOS (支持CUDA)

2. 软件环境

基础环境:

- Python 3.10 或更高版本
- CUDA 12.1+ (用于GPU加速)
- Conda (用于环境管理)

核心框架:

- Strands Agents (主框架)
- Ollama (本地LLM运行时)
- PyTorch (深度学习框架)
- Transformers (HuggingFace模型库)
- Diffusers (Stable Diffusion库)

3. 模型文件准备

需要准备以下训练好的模型:

模型1: 年画QA模型

- 基座: Qwen2.5-VL-3B-Instruct
- 微调方法: LoRA
- 训练数据: 2,220条QA对
- 存放位置: ./models/qwen2.5-vl-qa/

模型2: 用户标签模型

- 基座: Qwen3-4B-Instruct-2507
- 微调方法: LoRA
- 训练数据: 4,951个用户样本
- 存放位置: ./models/qwen3-user-tagging/

模型3: 推荐模型

- 基座: Qwen3-4B-Instruct
- 微调方法: LoRA
- 训练数据: 4,928个推荐样本
- 存放位置: ./models/qwen3-recommendation/

模型4: 图像生成模型

- 基座: Stable Diffusion 1.5
- LoRA权重: nianha_lora.safetensors (约50MB)
- 训练数据: 162张年画图片
- 存放位置: ./models/sd-nianha-lora/

💡 使用场景

场景1: 年画知识问答

用户输入: "请介绍一下杨柳青年画的特点"

系统处理流程:

1. 主协调器识别为知识问答任务
2. 调用 QA Agent
3. QA Agent加载 Qwen2.5-VL 模型
4. 根据训练的年画知识回答
5. 返回答案给用户

预期输出: 介绍杨柳青年画的工艺特点、色彩特征、题材内容等专业知识

场景2: 用户分析与推荐

用户输入: "我收藏了《门神》和《春牛图》,能推荐类似的吗?"

系统处理流程:

1. 主协调器识别为两步任务: 分析 + 推荐
2. 第一步: 调用标签Agent
    - 输入用户交互行为
    - 分析出风格和主题偏好
    - 输出: "风格:杨柳青, 主题:吉祥物"
3. 第二步: 调用推荐Agent
    - 输入用户画像和历史喜好
    - 匹配相似作品
    - 输出推荐列表和理由
4. 整合两步结果返回

预期输出:

- 用户偏好分析
- 3-5个推荐作品
- 每个作品的推荐理由

场景3: 图像生成

用户输入: "请生成一幅传统门神形象的年画,威武雄壮,红色调"

系统处理流程:

1. 主协调器识别为图像生成任务
2. 调用生成Agent
3. 构建完整prompt: "nianha, 传统门神, 威武雄壮, 红色调"
4. 使用Stable Diffusion + LoRA生成图像
5. 保存到 generated_images/ 目录
6. 返回图像路径

预期输出:

- 生成的年画图像文件
- 图像保存路径提示

场景4: 复杂组合任务

用户输入: "分析我的偏好,推荐作品,然后生成一张类似风格的新年画"

系统处理流程:

1. 主协调器识别为三步复合任务
2. 依次执行: 分析 → 推荐 → 生成
3. 每一步的输出作为下一步的输入
4. 整合三个环节的结果

体现优势:

- 主协调器自动理解任务拆解
- 无需手动编写工作流
- LLM驱动的智能编排